Universidad Autónoma de la Ciudad de México

Práctica 1: Resolución del problema de la mochila con algoritmos genéticos

Alumno:

Brian Miguel Escalona Maldonado

(21-003-1605)

1. Introducción

En está práctica se desarrolla la implementación de un algoritmo evolutivo para resolver el **problema de la mochila**. Este problema es un problema de optimización combinatoria. Dada una mochila con capacidad máxima de peso $W = 50$ y un conjunto de $n = 30$ objetos, cada uno con un peso w_1 y un valor v_1, el objetivo del problema es optimizar la selección de un subconjunto de objetos, buscando maximizar el valor total sin exceder la capacidad máxima de la mochila.

Primeramente, necesitamos la representación del individuo. Para esta práctica, se utilizo una representación binaria donde el valor 1 representa que el objeto se incluye en la mochila y el valor 0 representa que el objeto no se incluye en la mochila. Los objetos disponibles para ingresar a la mochila son los siguientes:

```
objetos = [
('Botella de agua', 30, 6),
('Comida deshidratada', 40, 8),
('Botiquín de primeros auxilios', 50, 10),
('Linterna LED', 28, 4),
('Baterías recargables', 14, 2),
('Cuchillo multiusos', 30, 5),
('Mapa topográfico', 15, 3),
('Brújula profesional', 16, 3),
('Tienda de campaña ultraligera', 70, 14),
('Saco de dormir térmico', 65, 13),
('Colchoneta inflable', 30, 6),
('Chamarra impermeable', 40, 9),
('Guantes térmicos', 15, 3),
('Filtro portátil de agua', 35, 5),
('Cuerda de escalada', 50, 12),
('Mosquetones de acero', 22, 5),
('Estufa portátil', 36, 8),
('Cartucho de gas', 20, 4),
('Kit de reparación', 18, 4),
('Radio de emergencia', 45, 9),
('Panel solar portátil', 65, 12),
('GPS de montaña', 42, 6),
('Cámara compacta', 40, 6),
('Cuaderno impermeable', 10, 2),
('Teléfono satelital', 40, 5),
('Laptop ultraligera', 110, 15),
('Power bank alta capacidad', 28, 4),
('Raciones energéticas', 28, 5),
('Manta térmica', 12, 2),
('Silbato de emergencia', 10, 2)
]
```

Esta lista con los 30 objetos contiene tres campos que agrupan los datos de la siguiente forma:

- **Primer campo:** Nombre del objeto
- **Segundo campo:** Valor del objeto
- **Tercer campo:** Peso del objeto

Ya que tenemos la representación y los objetos disponibles, generamos un individuo de manera aleatoria, tal y como se muestra en el siguiente campo:

```
[0, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
1, 1, 1, 0, 0, 0]
```

Este individuo representa una mochila en particular con distintos objetos dentro de ella. Cuando lo mapeamos de vuelta, nos queda una mochila con los siguientes objetos:

```
    ['Linterna LED',
    'Baterias recargables',
    'Cuchillo multiusos',
    'Mapa topografico',
    'Brujula profesional',
    'Guantes termicos',
    'Filtro portatil de agua',
    'Telefono satelital',
    'Laptop ultraligera',
    'Power bank alta capacidad']
```

Con lo anteriormente mencionado, podemos empezar a implementar las funciones necesarias para implementar el algoritmo genético que resuelva de manera correcta el problema de la mochila.

2. Desarrollo

Para esta práctica, en la *Tabla:* 1 se muestran los distintos parámetros y valores para el algoritmo genético. Esta tabla muestra todas las combinaciones que realizaremos para evaluar como estos parámetros afectan (de manera positiva o negativa) la convergencia de nuestro algoritmo.

Parámetro	Valor
Representación	Cadenas binarias
Recombinación	a) 1 punto b) 2 puntos c) Cruza uniforme
Probabilidad de recombinación	100 %
Mutación	a) Selección aleatoria de un solo gen b) Alterar cada gen independientemente con una probabilidad *tasa_mutacion*
Probabilidad de mutación (*tasa_mutacion*)	10 %
Selección de padres	Torneo: los 2 mejores de 5 padres aleatorios
Selección de supervivencia	Sin elitismo Con elitismo
Tamaño de la población	100 individuos
Número de descendientes	2 hijos
Inicialización de la población	Aleatoria
Función de aptitud	a) Fitness-Cero b) Lineal c) Cuadrática d) Exponencial
Condición de terminación	100 generaciones

Tabla 1: Parámetros del algoritmo genético

Se implementaron todas y cada una de las funciones para el correcto funcionamiento del algoritmo genético, (en el *Código:* 2 se muestra la implementación completa de las funciones). La parte más importante del algoritmo son las distintas funciones de penalización, estás son fundamentales pues se integran directamente a la función de aptitud, la cual se define de la siguiente manera:

$$fitness = valor_total - penalizacion$$

Esta penalización es la que determinara que tanto se penalizan a los individuos que se pasen del peso máximo $W = 50$, para esta práctica tenemos cuatro penalizaciones:

- **Fitness Cero**
- **Penalización Lineal**
- **Penalización Cuadrática**
- **Penalización Exponencial**

Estás penalizaciones van desde las más permisivas hasta algunas más severas. De manera inicial parecería que las penalizaciones más fuertes serían mejores. Sin embargo, ya iremos explorando como estas funciones afectan a nuestra aptitud y al final como esto afecta a los resultados de nuestro algoritmo.

Para realizar las ejecuciones se utilizo la biblioteca *itertools*. Con está biblioteca podemos calcular todas y cada una de las combinaciones de nuestros parámetros; según nuestra tabla de parámetros 1 tenemos:

- **Cruza: 3 variantes (1 punto, 2 puntos, Uniforme)**
- **Mutación: 2 variantes (Un gen, Independiente)**

- **Penalización: 4 variantes (Cero, Lineal, Cuadrática, Exponencial)**
- **Elitismo: 2 variantes (True, False)**

Esto nos da un total de configuraciones únicas: $3 \times 2 \times 4 \times 2 =$ **48 configuraciones**. Que se ejecutarán 100 veces de manera independiente, tal como pide la práctica; esto nos da un total de $48 \times 100 = 4,800$ ejecuciones. Para poder ejecutar todas esas configuraciones, se hizo uso de un servidor proporcionado por la universidad, utilizando **Temux**, que es una herramienta que permite gestionar múltiples sesiones de terminal en un solo espacio de trabajo y nos permite mantener tareas o procesos funcionando a pesar de que cerremos la terminal o se pierda la conexión con un servidor remoto; esto nos permitió dejar corriendo el proceso y realizar otras actividades, siendo una herramienta sumamente útil.

2.1. Análisis de los resultados

Después de la ejecución de nuestros algoritmos, generamos un archivo *.csv* con los resultados de cada una de las ejecuciones del algoritmo. A continuación, se presentan las configuraciones con la mejor aptitud generada por el algoritmo:

ID	Cruza	Mutacion	Penalizacion	Elitismo	Mejor Fitness	Diversidad Final	Peso Final	Valor Final
1	cruce_un_punto	mutacion_un_gen	fitness_lineal	True	912	0.00132	192	1054
2	cruce_un_punto	mutacion_independiente	fitness_lineal	True	904	0.18365333	190	1044
3	cruce_uniforme	mutacion_independiente	fitness_lineal	True	900	0.18296	190	1040

Tabla 2: Configuraciones con mejor fitness

Como podemos ver en la tabla anterior, estas configuraciones cuentan con individuos que tienen una aptitud demasiado alta y un valor final demasiado alto. Sin embargo, tenemos un detalle: el peso final está muy por encima del peso máximo. Esto es un problema, ya que al mirar al individuo generado, podremos ver que está lleno de 1s. Esto significa que el algoritmo nos devuelve una mala solución.

```
[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
['Botella de agua', 'Comida deshidratada', 'Botiquin de primeros auxilios', 'Linterna LED', 'Baterias recargables', 'Cuchillo multiusos', 'Mapa topografico', 'Brujula profesional', 'Tienda de campaña ultraligera', 'Saco de dormir termico', 'Colchoneta inflable', 'Chamarra impermeable', 'Guantes termicos', 'Filtro portatil de agua', 'Cuerda de escalada', 'Mosquetones de acero', 'Estufa portatil', 'Cartucho de gas', 'Kit de reparacion', 'Radio de emergencia', 'Panel solar portatil', 'GPS de montana', 'Camara compacta', 'Cuaderno impermeable', 'Telefono satelital', 'Laptop ultraligera', 'Power bank alta capacidad', 'Raciones energeticas', 'Manta termica', 'Silbato de emergencia'
```

Como podemos ver, la mochila devuelta por el algoritmo es una mala solución considerando la restricción de peso máximo; sin embargo, el algoritmo la devuelve por tener una buena aptitud. En las *Figuras* 1 y 2 podemos ver como la aptitud promedio y máxima casi no evoluciona a través de las generaciones, quedándose estancada prácticamente desde la generación número 20, mientras que la diversidad de la población va disminuyendo hasta prácticamente 0 antes de llegar siquiera a la generación 20. Esto es un problema, ya que nuestro algoritmo se quedo con soluciones que, a pesar de tener una buena aptitud, tienen un peso muy por encima del esperado. Pero, ¿Por qué pasa esto? Primero que nada, debemos recordar la manera en que está definida está penalización. La formula es la siguiente:

$$Penalizacion = \begin{cases} 0, & \text{si } peso_total \leq W, \\ \alpha \cdot (peso_total - W), & \text{si } peso_total > W. \end{cases}$$

Esto quiere decir que en el caso de nuestro individuo, sustituyendo valores nuestra aptitud queda de esta manera:

$$Penalizacion = 1 \cdot (192 - 50) = 142$$

$$fitness = valor_total - Penalizacion = 1054 - 142 = 912$$

Como podemos observar aún con la penalización el individuo está bien calificado, por lo que está penalización poco castiga a los malos individuos, permitiendo la cruza de los mismos.

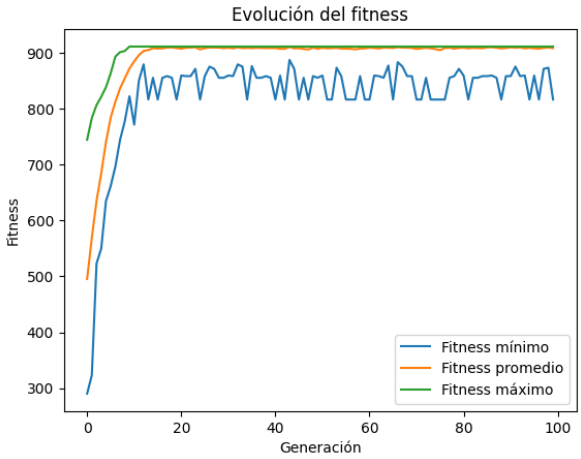


Figura 1: Evolución de la aptitud a través de las épocas

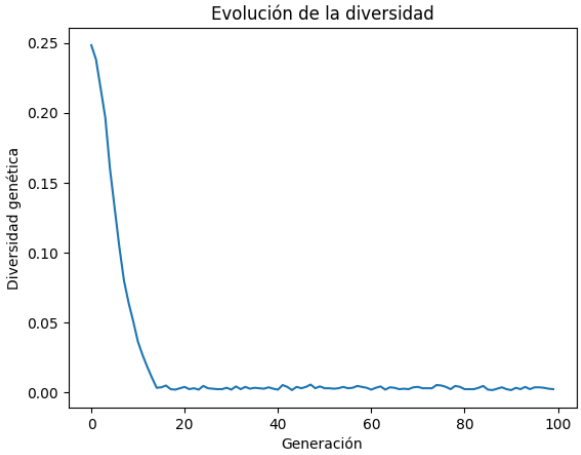
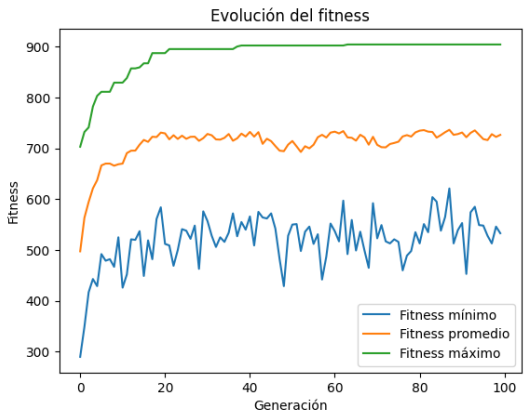
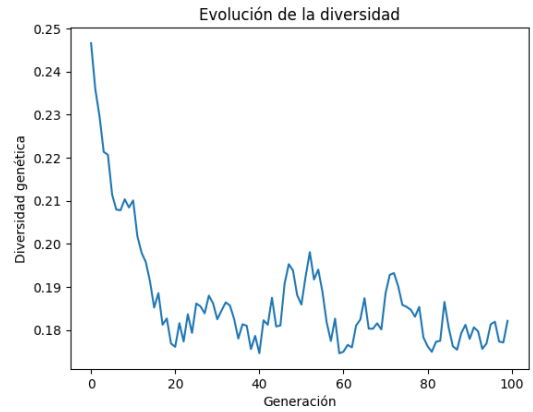


Figura 2: Evolución de la diversidad a través de las épocas

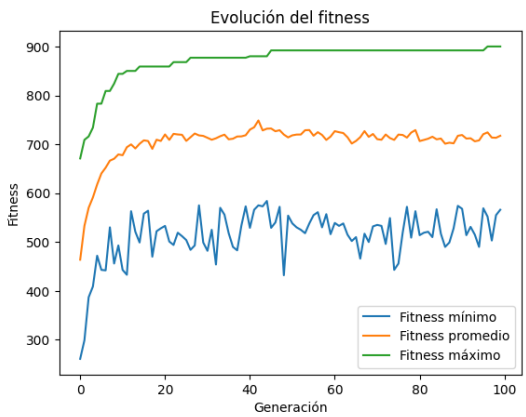
Si analizamos las gráficas generadas por las otras dos configuraciones, tenemos una evolución de la aptitud bastante parecida, con una aptitud máxima cercana a los 900 y saltos bastante agresivos en la aptitud mínima. Mientras que la aptitud media se mantiene bastante estable, cerca de los valores de 600 y 700; por otro lado, la diversidad genética presentada en la *Figura* 2 evoluciona rápidamente hacia cero, mientras que las presentadas en las *Figuras:* 3b y 3d mantienen una diversidad más variada, lo que permite buscar mejores soluciones; sin embargo, dada la penalización tan permisiva, no podemos encontrar una buena solución.



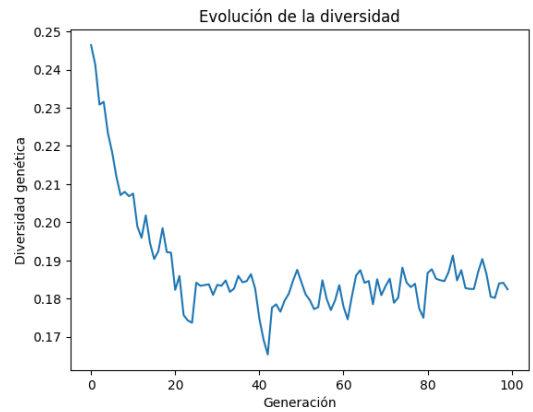
(a) Evolución de la aptitud 2



(b) Evolución de la diversidad 2



(c) Evolución de la aptitud 3



(d) Evolución de la diversidad 3

Figura 3: Comparación de la aptitud y diversidad

2.2. Solución al problema de peso

Para enfrentar este problema, se filtraron los datos a partir de dos columnas importantes: la columna con la mejor aptitud y la columna de peso final, filtrando de esta manera solamente a los individuos que tuvieran un peso $W \leq 50$. De esta manera, obtuvimos nuevos individuos que, a pesar de no contar con una aptitud espectacular, si cumplían con la restricción del peso, lo cual, en este contexto, es de suma importancia, pues una mochila que excede el peso no es una solución optima. En la *Tabla:* 3 se muestran las configuraciones que cumplen con el peso establecido y con la mejor aptitud.

ID	Cruza	Mutacion	Penalizacion	Elitismo	Mejor Fitness	Diversidad Final	Peso Final	Valor Final
1	cruce_uniforme	mutacion_independiente	fitness_cero	True	351	0.216403	50	351
2	cruce_dos_puntos	mutacion_un_gen	fitness_cero	True	347	0.00295	50	347
3	cruce_un_punto	mutacion_independiente	fitness_exponencial	False	345.96	0.213123	50	350

Tabla 3: Configuraciones con peso en el rango

Como podemos ver en la tabla anterior, la aptitud de estos individuos son ”peores” comparadas a las generadas anteriormente. Sin embargo, podemos observar que todos los individuos cumplen con la condición de llevar el peso máximo permitido, devolviendo una mochila con un buen valor y completamente llena.

```
[0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0,
1, 1, 0, 0, 1, 0]
['Linterna LED', 'Baterias recargables', 'Cuchillo multiusos', 'Filtro
portatil de agua', 'GPS de montana', 'Camara compacta', 'Telefono
satelital', 'Laptop ultraligera', 'Manta termica']
```

Como podemos ver en este caso, la mochila está llena de objetos con un buen valor; esto es una mejora respecto a lo que vimos anteriormente con la penalización lineal. En cambio, como podemos ver en la *Figura* 4 el uso de la penalización `fitness_cero` obliga a que nuestra aptitud miníma se mantenga en cero. De esta manera, también obligamos a que la aptitud promedio se mantenga baja; con esto, y con el **elitismo** activado, logramos que los mejores individuos se mantengan siempre a través de las generaciones. Esto podemos verlo en la gráfica, con la aptitud máxima que se conserva a través de las generaciones; con esto aseguramos que siempre tenemos una solución optima. La *Figura:* 5 muestra como en este caso la diversidad varia demasiado, lo que pudo haber ayudado a que nuestro algoritmo encontrará una buena solución, quedando por encima de las otras configuraciones.

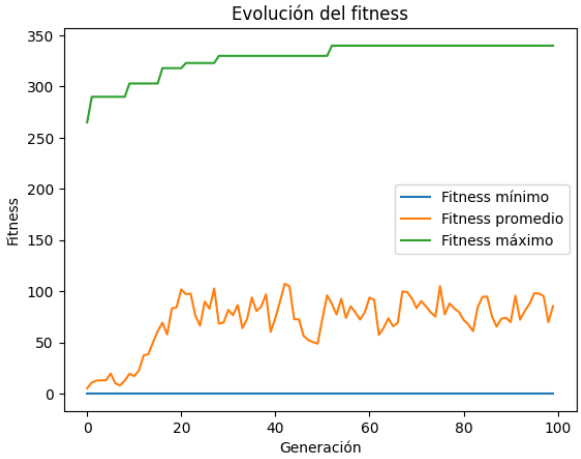


Figura 4: Evolución de la aptitud a través de las épocas

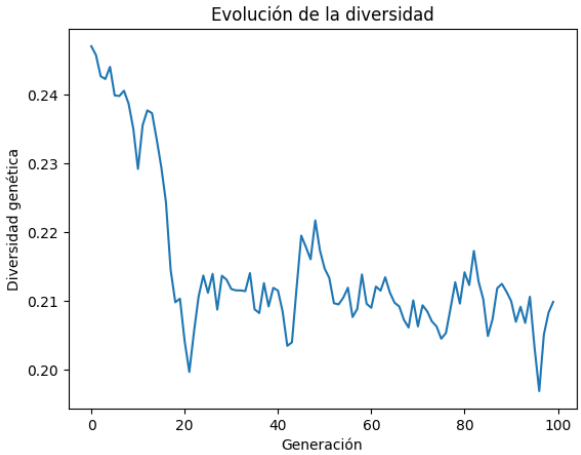
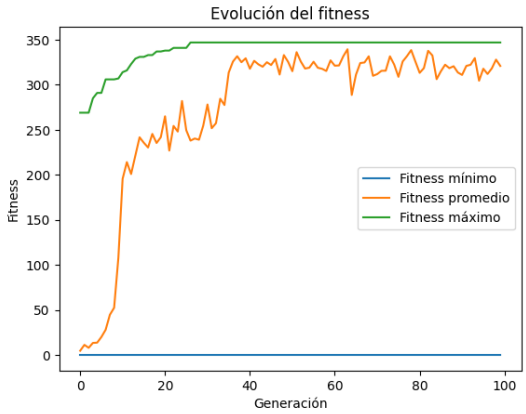
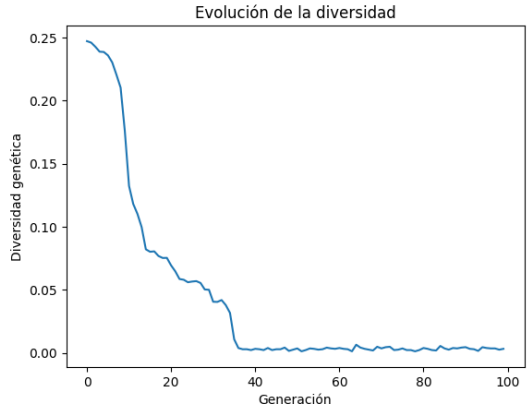


Figura 5: Evolución de la diversidad a través de las épocas

En la *Figura:* 6 se muestran las gráficas de las otras dos configuraciones; ambas tuvieron el mismo peso. Lo único que las diferencia es su aptitud, que es de 5 o 6 puntos, lo cual no es demasiado. Como podemos ver en la *Figura:* 6b la evolución de la aptitud máxima se mantuvo durante varias generaciones, por lo que podemos observar que, a pesar de que la aptitud promedio es bastante mayor, la diversidad de los individuos disminuye rápidamente. Esta es la razón por la que el algoritmo en algún momento se estanco en un mismo valor máximo. Otro aspecto a destacar es que la diferencia entre los individuos devueltos por la primera configuración y la segunda configuración es que son muy parecidos, pues solo difieren en un objeto; está es la razón por la cual ambos tienen aptitudes muy parecidas.

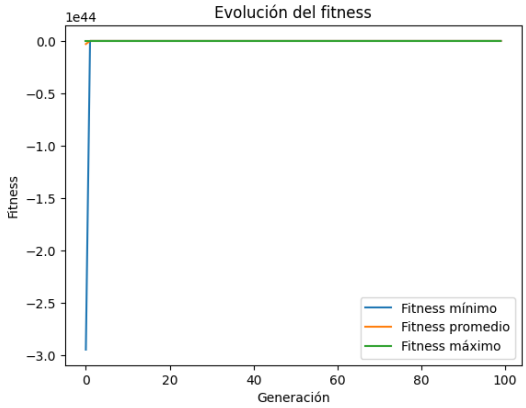


(a) Evolución de la aptitud 5

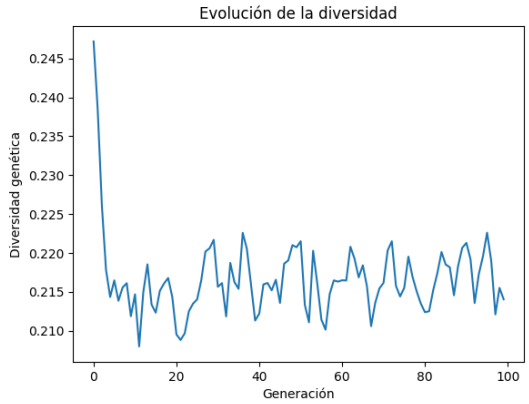


(b) Evolución de la diversidad 5

```
    [0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1, 1, 0, 0, 1, 1]
    ['Linterna LED', 'Cuchillo multiusos', 'Filtro portatil de agua', 'GPS de montana', 'Camara compacta', 'Telefono satelital', 'Laptop ultraligera', 'Manta termica', 'Silbato de emergencia']
```



(c) Evolución de la aptitud 6



(d) Evolución de la diversidad 6

```
    [0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0]
    ['Baterias recargables', 'Cuchillo multiusos', 'Mapa topografico', 'Chamarra impermeable', 'Filtro portatil de agua', 'Cartucho de gas', 'Kit de reparacion', 'Telefono satelital', 'Laptop ultraligera', 'Power bank alta capacidad']
```

Figura 6: Comparación de la aptitud y diversidad

Lo anterior no pasa con la tercera configuración, pues está es la más interesante de todas, como podemos ver en la *Figura:* 6c. Parece que todas las métricas (aptitud máxima, aptitud media y aptitud mínima) están en el mismo valor (cero); sin embargo, esto se debe a que la **penalización exponencial** genera valores muy irregulares, los cuales la gráfica no puede representar; en la salida presentada en: 1, se muestran algunos de estos valores. Como podemos observar, los valores tienen demasiadas oscilaciones entre valores muy pequeños ($-865934004239498{,}8$, por ejemplo), hasta valores más altos; lo mismo ocurre con los valores de la aptitud media, que van desde algunos muy pequeños ($-69373098697429{,}6$

hasta $-1{,}899408162036739e+16$); es por esa razón que la gráfica no puede representar estos valores tan irregulares que produce la penalización. Pero, ¿Por qué pasa esto?

Recordemos primero como se define la **penalización exponencial**:

$$Penalizacion = \begin{cases} 0, & \text{si } peso_total \leq W, \\ \alpha \cdot e^{(peso_total - W)}, & \text{si } peso_total > W \end{cases}$$

Por ejemplo, utilizando nuestro primer individuo generado por nuestro algoritmo, que contaba con un peso $W = 192$ y un valor total de 1054, la aptitud quedaría de la siguiente forma:

$$Penalizacion = 0{,}01 \cdot e^{192-50} = 0{,}01 \cdot e^{142} = 6{,}2 \times 10^{61}$$

$$fitness = valor_total - Penalizacion = 1054 - 6{,}2 \times 10^{61} = -6{,}2 \times 10^{61}$$

Como podemos ver en este caso, la penalización es extrema; el valor del orden de -10^{61} es sumamente bajo. Este tipo de penalización es una de las más fuertes y extremas de todas, pero, como podemos ver, da resultados. Sin embargo, considerando el costo computacional que requieren estos cálculos, puede que no valga la pena teniendo en cuenta penalizaciones como **fitness_cero** que es igual de efectiva y menos costosa.

```
Mínima:
np.float64(-3.493427105748496e+17), np.float64(-865934004239426.8), np.float64
    (-865934004239498.8), np.float64(-7.016735912097614e+18), np.float64
    (-3.1042979357019095e+24), np.float64(-1.907346572495095e+19), np.float64
    (-4.727839468229288e+16), np.float64(-1.907346572495095e+19),

Media:
    np.float64(-1.899408162036739e+16), np.float64(-1.9073703167944467e+17),
    np.float64(-69373098697429.6), np.float64(-38855253086451.0), np.float64
    (-4.205232213823367e+21), np.float64(-1356723522096039.2), np.float64
    (-5.184752390363539e+17), np.float64(-9520950721233610.0),


Máxima:
    np.float64(322.96571206507264), np.float64(329.96571206507264), np.float64
    (318.96571206507264), np.float64(332.0336684157154), np.float64
    (337.96571206507264), np.float64(323.51586840897426), np.float64
    (333.0336684157154),
```

Código 1: Salida de la aptitud mínima, media y máxima, configuración 3

3. Conclusiones

El desarrollo de esta práctica ha resultado en varias conclusiones interesantes; primeramente, sirvió para reforzar los conocimientos adquiridos anteriormente en clase. Esto fue beneficioso, ya que el desarrollo de las funciones para las distintas penalizaciones, métodos de cruce y selección de supervivientes me permitió entender de mejor manera la forma en la que funcionan estos algoritmos.

Uno de los descubrimientos más importantes es cómo afecta la función de aptitud a nuestro algoritmo; esto puede ser de manera positiva o negativa. El caso sucedido con la aptitud usando la **penalización lineal** es bastante interesante, pues podemos ver que, a pesar de que teóricamente suena como la opción más viable (ya que no castiga de manera tan severa a los malos individuos), en una primera instancia sonaba como la mejor opción; sin embargo, después de mirar los resultados, estos mostraron un comportamiento opuesto. La **penalización lineal** al ser permisiva, da paso a que se reproduzcan malos individuos y, a pesar de exceder el peso establecido $W = 50$ siguen considerándose buenas soluciones. En este aspecto, la penalización que presento el mejor rendimiento fue la **fitness_cero**; esta resulto ser perfecta para nuestro problema, pues permite a nuestro algoritmo evolucionar de manera correcta. Ya que, al obligar a la aptitud a ser cero en todos los casos en los que el peso excede el máximo, esto guía al algoritmo hacia regiones factibles del espacio de búsqueda para evolucionar.

Otro aspecto importante en el desarrollo de nuestra práctica es el **Elitismo**; este tipo de selección de sobrevivientes es una manera de garantizar que siempre se tiene un individuo con buena aptitud; esto ayuda a nuestro algoritmo.

Otro aspecto importante que se presenta en las soluciones con buena aptitud es el mecanismo de **mutación independiente**; este mecanismo es importante porque nos permite explorar mayores regiones de nuestro campo de búsqueda, ya que mutar un único gen solo permite explorar pocas regiones, dado que el individuo mutado se queda muy cerca del individuo original, En contraste, la mutación independiente permite que cada gen pueda alterarse con cierta probabilidad. Esto genera individuos que pueden diferir significativamente del original, favoreciendo la exploración del espacio de búsqueda. Esto nos permite mantener una mayor diversidad genética, tal y como podemos ver en las *Figuras:* 3b 3d 5 6d.

Estos resultados son importantes, ya que nos permiten entender mejor que estrategias funcionan en el desarrollo de un algoritmo evolutivo, para de esta manera poder aplicarlas a futuros trabajos en los que se opte por representaciones binarias.

Por otro lado, el análisis de las gráficas de aptitud permitió observar el fenómeno de la convergencia prematura en ciertas configuraciones. Se identificó que, sin el uso de elitismo, el algoritmo tiende a presentar oscilaciones en la aptitud máxima, lo que ralentiza la búsqueda de la solución óptima. En contraste, al combinar el elitismo con la cruza uniforme, el algoritmo mostró una capacidad superior para combinar esquemas de alta calidad de ambos padres, acelerando la aparición de individuos que rozan el límite de capacidad de la mochila $W = 50$ con valores de aptitud sobresalientes.

Finalmente, es posible afirmar que el éxito de un algoritmo genético para el problema de la mochila no reside en un único operador, sino en el equilibrio entre la explotación (preservar los mejores genes mediante elitismo y selección por torneo) y la exploración (introducir variabilidad mediante mutación independiente). La experiencia obtenida subraya que, en problemas con restricciones rígidas de peso, las penalizaciones severas como **fitness_cero** son preferibles, ya que delimitan claramente las fronteras del espacio de búsqueda factible, evitando que el algoritmo desperdicie recursos computacionales en regiones inviables.

Estos resultados son fundamentales, ya que no solo resuelven el problema planteado, sino que proporcionan un marco de referencia metodológico para ajustar parámetros en otros problemas de optimización combinatoria de mayor complejidad.

4. Anexo

4.1. Implementación de funciones para algoritmo genético

```python
    import pandas as pd
    import numpy as np
    import itertools
    import random

    SEED = 64

    np.random.seed(SEED)
    random.seed(SEED)

    """
    Sintaxis de los datos: Es una lista que contiene elementos de tipo tupla
    que
    agrupa los datos de la siguiente forma:

    Primer campo: Nombre del objeto.
    Segundo campo: Valor del objeto.
    Tercer campo: Peso del objeto.
"""

    objetos = [
    ('Botella de agua', 30, 6),
    ('Comida deshidratada', 40, 8),
    ('Botiquin de primeros auxilios', 50, 10),
    ('Linterna LED', 28, 4),
    ('Baterias recargables', 14, 2),
    ('Cuchillo multiusos', 30, 5),
    ('Mapa topografico', 15, 3),
    ('Brujula profesional', 16, 3),
    ('Tienda de campaña ultraligera', 70, 14),
    ('Saco de dormir termico', 65, 13),
    ('Colchoneta inflable', 30, 6),
    ('Chamarra impermeable', 40, 9),
    ('Guantes termicos', 15, 3),
    ('Filtro portatil de agua', 35, 5),
    ('Cuerda de escalada', 50, 12),
    ('Mosquetones de acero', 22, 5),
    ('Estufa portatil', 36, 8),
    ('Cartucho de gas', 20, 4),
    ('Kit de reparacion', 18, 4),
    ('Radio de emergencia', 45, 9),
    ('Panel solar portatil', 65, 12),
    ('GPS de montana', 42, 6),
    ('Camara compacta', 40, 6),
    ('Cuaderno impermeable', 10, 2),
    ('Telefono satelital', 40, 5),
    ('Laptop ultraligera', 110, 15),
    ('Power bank alta capacidad', 28, 4),
    ('Raciones energeticas', 28, 5),
    ('Manta termica', 12, 2),
    ('Silbato de emergencia', 10, 2)
    ]

    def obtener_datos(indice, nombre=False, valor=False, peso=False):
        if nombre:
            return objetos[indice][0]
        if valor:
```

```
            return objetos[indice][1]
        if peso:
            return objetos[indice][2]
        else:
            return objetos[indice]

    def generar_poblacion(n_objetos: int = 30, tam_poblacion: int =100):
        poblacion = np.random.randint(0, 2, (tam_poblacion, n_objetos))
        return poblacion

    def calcular_valor_total(individuo):
        valor_total = []
        for _, i in enumerate(individuo):
            valor = obtener_datos(_, valor=True)
            valor_total.append(i * valor)
        return np.sum(valor_total)


    def calcular_peso_total(individuo):
        peso_total = []
        for _, i in enumerate(individuo):
            peso = obtener_datos(_, peso=True)
            peso_total.append(i * peso)
        return np.sum(peso_total)


    def fitness_cero(poblacion, peso_maximo: int = 50):
        fitness = []
        for p in poblacion:
            valor = calcular_valor_total(p)
            peso = calcular_peso_total(p)
            if peso > peso_maximo:
                fitness.append(0)
            else:
                fitness.append(valor)
        return fitness

    def fitness_lineal(poblacion, peso_maximo: int = 50):
        fitness = []
        factor_penalizacion: int = 1
        penalizacion: int = -1
        for p in poblacion:
            valor = calcular_valor_total(p)
            peso = calcular_peso_total(p)
            if peso > peso_maximo:
                 penalizacion = factor_penalizacion * (peso - peso_maximo)
            if peso <= peso_maximo:
                penalizacion = 0
            fitness.append(valor - penalizacion)
        return fitness

    def fitness_cuadratico(poblacion, peso_maximo: int = 50):
        fitness = []
        factor_penalizacion: float = 0.2
        penalizacion: float = -1
        for p in poblacion:
            valor = calcular_valor_total(p)
            peso = calcular_peso_total(p)
            if peso > peso_maximo:
                 penalizacion = factor_penalizacion * ((peso - peso_maximo) **
 2)
            if peso <= peso_maximo:
```

```
                penalizacion = 0
            fitness.append(valor - penalizacion)
        return fitness

    def fitness_exponencial(poblacion, peso_maximo: int = 50):
        fitness = []
        factor_penalizacion: float = 0.01
        penalizacion: float = -1
        for p in poblacion:
            valor = calcular_valor_total(p)
            peso = calcular_peso_total(p)
            if peso > peso_maximo:
                 penalizacion = factor_penalizacion * np.e ** (peso - peso_
maximo)
            if peso <= peso_maximo:
                penalizacion = 0
            fitness.append(valor - penalizacion)
        return fitness

    def seleccionar_padres(poblacion, fitness, n_padres=2, n_torneo=5):

        indices_torneo = np.random.choice(len(poblacion), size=n_torneo,
replace=False)

        candidatos = []
        for idx in indices_torneo:
            candidatos.append((poblacion[idx], fitness[idx]))

        candidatos_ordenados = sorted(candidatos, key=lambda x: x[1], reverse=
True)

        ganadores = [c[0] for c in candidatos_ordenados[:n_padres]]

        return ganadores

    def cruce_un_punto(padres):

        padre_1 = padres[0]
        padre_2 = padres[1]

        n = len(padre_1)

        punto = np.random.randint(1, n - 1)

        hijo1 = np.concatenate([padre_1[:punto], padre_2[punto:]])
        hijo2 = np.concatenate([padre_2[:punto], padre_1[punto:]])

        return hijo1, hijo2

    def cruce_dos_puntos(padres):

        padre_1 = padres[0]
        padre_2 = padres[1]

        n = len(padre_1)

        puntos = sorted(np.random.choice(range(1, n), size=2, replace=False))
        p1, p2 = puntos

        hijo1 = np.concatenate([padre_1[:p1], padre_2[p1:p2], padre_1[p2:]])
        hijo2 = np.concatenate([padre_2[:p1], padre_1[p1:p2], padre_2[p2:]])

```

```python
        return hijo1, hijo2

    def cruce_uniforme(padres, prob_intercambio=0.10):

        padre_1 = padres[0]
        padre_2 = padres[1]

        n = len(padre_1)
        hijo1 = np.zeros(n, dtype=int)
        hijo2 = np.zeros(n, dtype=int)

        for i in range(n):
            if np.random.rand() < prob_intercambio:
                hijo1[i], hijo2[i] = padre_2[i], padre_1[i]
            else:
                hijo1[i], hijo2[i] = padre_1[i], padre_2[i]

        return hijo1, hijo2

    def mutacion_un_gen(individuo, tasa_mutacion: int = 0.10):

        n = len(individuo)

        punto = np.random.randint(1, n - 1)
        if np.random.random() < tasa_mutacion:
            bit = individuo[punto]
            bit_mutado = 1 - bit
            individuo[punto] = bit_mutado
        return individuo

    def mutacion_independiente(individuo, tasa_mutacion: int = 0.10):
        for i, bit in enumerate(individuo):
            if np.random.random() < tasa_mutacion:
                bit = individuo[i]
                bit_mutado = 1 - bit
                individuo[i] = bit_mutado

        return individuo

    def calcular_diversidad(poblacion):

        p_promedios = np.mean(poblacion, axis=0)
        varianzas = np.mean((poblacion - p_promedios)**2, axis=0)
        return np.mean(varianzas)

    def algoritmo_genetico(metodo_cruza, metodo_mutacion, tipo_penalizacion,
    con_elitismo):

        tamano_poblacion = 100
        max_generaciones = 100
        tasa_mutacion = 0.10
        peso_maximo = 50


        historial = {'min': [], 'max': [], 'mean': [], 'div': []}

        poblacion = generar_poblacion()

        for generacion in range(max_generaciones):

            aptitudes = tipo_penalizacion(poblacion, peso_maximo)

```

```python


            historial['min'].append(np.min(aptitudes))
            historial['max'].append(np.max(aptitudes))
            historial['mean'].append(np.mean(aptitudes))
            historial['div'].append(calcular_diversidad(poblacion))

            nueva_poblacion = []

            mejor_individuo = poblacion[np.argmax(aptitudes)].copy()

            for _ in range(tamano_poblacion // 2):
                padres = seleccionar_padres(poblacion, aptitudes)

                h1, h2 = metodo_cruza(padres)

                h1 = metodo_mutacion(h1, tasa_mutacion)
                h2 = metodo_mutacion(h2, tasa_mutacion)

                nueva_poblacion.extend([h1, h2])

            if con_elitismo:
                nueva_poblacion[0] = mejor_individuo

            poblacion = np.array(nueva_poblacion)

        aptitudes = tipo_penalizacion(poblacion, peso_maximo)

        mejor_idx = np.argmax(aptitudes)
        mejor_individuo = poblacion[mejor_idx]

        peso_final = calcular_peso_total(mejor_individuo)
        valor_final = calcular_valor_total(mejor_individuo)

        print(f"Mejor Fitness: {aptitudes[mejor_idx]}")
        print(f"Peso Total: {peso_final} (Límite: 50)")
        print(f"Valor Total: {valor_final}")

        return mejor_individuo, historial

    configuraciones = {
        'cruza': [cruce_un_punto, cruce_dos_puntos, cruce_uniforme],
        'mutacion': [mutacion_un_gen, mutacion_independiente],
        'penalizacion': [fitness_cero, fitness_lineal, fitness_cuadratico,
    fitness_exponencial],
        'elitismo': [True, False]
    }


    resultados_totales = []

    combinaciones = list(itertools.product(
        configuraciones['cruza'],
        configuraciones['mutacion'],
        configuraciones['penalizacion'],
        configuraciones['elitismo']
    ))

    print(f"Iniciando experimentos. Total de configuraciones a probar: {len(
    combinaciones)}")

    for cruza, mutacion, penalizacion, elitismo in combinaciones:
```

```python
        nombre_config = f"{cruza.__name__} | {mutacion.__name__} | {penalizacion.__name__} | Elitismo:{elitismo}"
        print(f"Ejecutando: {nombre_config}")


        for i in range(100):

            mejor_ind, historial = algoritmo_genetico(cruza, mutacion, penalizacion, elitismo)


            mejor_fitness_ejecucion = historial['max'][-1]

            resultados_totales.append({
                'Configuracion': nombre_config,
                'Cruza': cruza.__name__,
                'Mutacion': mutacion.__name__,
                'Penalizacion': penalizacion.__name__,
                'Elitismo': elitismo,
                'Ejecucion_No': i + 1,
                'Mejor_Fitness': mejor_fitness_ejecucion,
                'Diversidad_Final': historial['div'][-1],
                'Peso_Final': calcular_peso_total(mejor_ind),
                'Valor_Final': calcular_valor_total(mejor_ind)
            })

    df_resultados = pd.DataFrame(resultados_totales)
    df_resultados.to_csv('resultados_practica_mochila.csv', index=False)

    print("¡Proceso completado! Archivo 'resultados_practica_mochila.csv' generado.")
```

Código 2: Código del algoritmo genético

4.2. Funciones para graficar

```python

    import matplotlib.pyplot as plt

    def graficar_fitness(historial):

        plt.plot(historial['min'], label="Fitness mínimo")
        plt.plot(historial['mean'], label="Fitness promedio")
        plt.plot(historial['max'], label="Fitness máximo")

        plt.xlabel("Generación")
        plt.ylabel("Fitness")
        plt.title("Evolución del fitness")
        plt.legend()

        plt.show()
    def graficar_diversidad_genetica(historial):
        plt.plot(historial['div'])

        plt.xlabel("Generación")
        plt.ylabel("Diversidad genética")
        plt.title("Evolución de la diversidad")

        plt.show()
```

Código 3: Funciones de graficación